

BareCode

Offline supply-chain X-ray for Python environments. Zero dependencies.

Team Import Error (solo) · **Event** Zero Dependency Hackathon 2026 · **Track** A — Developer Tools & CLI
Language Python 3.14 (standard library only) · **Repo** github.com/Deven-Kulthia/barecode · **Licence** MIT

1. What this document is

A complete explanation of what we built, why, in what language, how it works, and what we claim — written so you can defend every part of it in writing to the judges. Judging is asynchronous: judges score from the repo and the video, then may email questions. Nothing here is a claim I have not actually run.

2. The one-sentence pitch

BareCode tells you, offline and in seconds, what is really inside a Python environment — and whether the installed files still match the hashes the installer recorded. It replaces four tools you would normally `pip install`, and it has an empty dependency manifest itself.

3. The problem we solve

Every Python wheel you install writes a `RECORD` file: a CSV listing the SHA-256 of every file it placed on disk. **Nothing ever checks it again.**

Existing tool	What it actually checks
<code>pip check</code>	Dependency <i>metadata</i> consistency. Not file contents.
<code>pip install --require-hashes</code>	The downloaded archive, at install time only. Never again.
<code>pip-audit</code>	Installed <i>versions</i> against a vulnerability database. Needs network.
There is no <code>pip verify</code>	— this is the gap.

So if a file inside `site-packages` changes after installation — a poisoned post-install step, a patched file, a compromised mirror, a shared environment someone "fixed" — the standard toolchain is completely silent.

The hackathon's own front page is built on exactly these incidents: left-pad (2016), the xz backdoor (2024), the chalk/debug crypto-clipper (2025), the Shai-Hulud worm, and **ChainDrop (August 2026)** — roughly 444 packages poisoned in under four hours *with valid build provenance*. The metadata was correct because it genuinely was. Only re-hashing the bytes on disk catches that.

4. Why Python, and not JavaScript, Go or Rust

The decision had three inputs: what the rules allow, what suits the project, and what you can defend in writing.

Consideration	Outcome
Is Python allowed for Track A?	Yes, explicitly. The rules list Python with its own dependency rules, and Track A's official guidance names <code>argparse</code> and <code>pathlib / os.walk</code> as things that "carry" the track.
Is Python <i>good</i> for a CLI?	Better than Node here. <code>argparse</code> has subcommands, type coercion, <code>choices</code> , counted flags and auto-generated help. Node's <code>util.parseArgs</code> has none of those — the official Node cheat sheet says so.
Rust?	Rejected. Its <code>std</code> has no JSON, no crypto, no compression, no async runtime. We would have rebuilt the standard library before writing a feature.
Go?	Genuinely strong, but not installed on this machine, and you would have had to defend Go code in written follow-up.
JavaScript?	You told me you cannot defend it. The rules require the team be reachable for written follow-up, and state that work which cannot be defended in writing "scores accordingly". Learning JS in the remaining time would have consumed the entire budget.

Python's one real weakness was the *dependency proof*: an absent `requirements.txt` proves nothing, whereas `"dependencies": {}` in a `package.json` is self-evident. Section 8 explains how we turned that into our strongest artifact instead.

5. What the tool does — five commands

Command	Question it answers
<code>barcode audit</code>	What is installed, who installed it, what came from outside a package index, what the licences are, and whether anything executes code at interpreter startup.
<code>barcode verify</code>	Do the installed files still match the hashes the installer recorded? This is the headline feature.
<code>barcode why</code> <code><pkg></code>	Why is this package here? Every dependency path, shortest first. With <code>--blast</code> , everything that breaks if it is compromised.
<code>barcode deps</code>	Do declared, installed and actually-imported dependencies agree? Reports <i>missing</i> , <i>unused</i> and <i>phantom</i> .
<code>barcode killable</code>	Which installed packages the standard library could replace — and, honestly, which it could not.

Everything runs fully offline. No network calls, ever. Nothing from the audited environment is imported or executed — it is all file reads.

Two findings that are genuinely under-examined

- `.pth` **startup hooks**. A `.pth` file in site-packages whose line begins `import` is executed by Python's `site` module on *every interpreter startup*. It is Python's equivalent of an npm `postinstall` hook, far less well known, and almost nobody audits it. We detect and report them — and never run them.
- `direct_url.json` **provenance**. Reveals packages installed from a git URL or a local path rather than from an index — code with no registry provenance sitting in your environment.

6. Architecture

```
cli.py      argparse subcommands, exit codes, JSON/human rendering
├── env.py   scan .dist-info: METADATA (email.parser), INSTALLER,
            direct_url.json provenance, .pth startup hooks, and the
            import-name -> distribution map derived from RECORD
├── integrity.py re-hash RECORD entries (csv + hashlib.file_digest,
            parallelised with concurrent.futures)
├── graph.py  PEP 508 requirement parsing, BFS `why` + blast radius
├── project.py declared deps (tomllib + requirements parser) vs actual
            imports (ast), and the three-way comparison
├── advisor.py curated package -> stdlib table with confidence levels
└── ansi.py  SGR styling, NO_COLOR/TTY precedence, width measurement

tools/prove_zero_deps.py the dependency proof (ast + sys.stdlib_module_names)
tests/test_barecode.py   79 tests over synthetic .dist-info fixtures
.github/workflows/ci.yml the same proof, on a clean runner, no pip install
```

Data flows one way: **scan** → **model** → **analyse** → **render**. Analyzers never touch the filesystem directly and renderers never compute anything, which is why the `--json` and human output paths cannot disagree with each other.

How `verify` works, step by step

1. Find the target `site-packages` from `--path` (a project dir, a virtualenv root, or site-packages itself).
2. For each `*.dist-info` directory, read `RECORD` with the `csv` module.
3. Each row is `path,sha256=<urlsafe-base64>,size`. Rows without a hash (`RECORD` itself, most `.pyc` files) are counted, not checked — we report that count so the coverage claim is honest.
4. Resolve the path and confirm it stays inside the **environment root**. A poisoned `RECORD` could contain `../../../../etc/passwd`; that is reported, never followed.
5. If the recorded size already differs, report `size-mismatch` without hashing.
6. Otherwise stream the file through `hashlib.file_digest`, encode as urlsafe base64 with padding stripped, and compare. A mismatch is `modified`.
7. Hashing runs on a `ThreadPoolExecutor` sized from `os.process_cpu_count()`. `file_digest` releases the GIL while reading, so the threads give real parallelism.

7. Zero dependencies — what we replaced

Full detail with rationale and inherited limitations is in `STDLIB.md` (17 substitutions). The headline ones:

Normally you install	We used instead
<code>click / typer</code>	<code>argparse</code> with subparsers
<code>colorama / rich</code>	raw ANSI SGR escapes, honouring <code>NO_COLOR</code> and TTY detection
<code>wcwidth</code>	a 20-line escape-skipping width function
<code>packaging</code> (METADATA)	<code>email.parser</code> — wheel METADATA genuinely <i>is</i> RFC 822
<code>packaging</code> (PEP 503 names)	one <code>re.sub</code> — that is the entire specification
<code>packaging</code> (PEP 508 reqs)	a reduced verbose regex
<code>networkx</code>	two dicts and a <code>collections.deque</code> BFS
<code>tar</code> -style CSV handling	<code>csv</code> — RECORD paths containing commas are quoted
<code>deptry / pipreqs</code>	<code>ast</code> walking <code>Import</code> / <code>ImportFrom</code>
<code>toml / tomli</code>	<code>tomllib</code> (it <i>is</i> tomli, vendored into the stdlib)
<code>pytest</code>	<code>unittest</code> — 79 tests
<code>pyinstaller / shiv</code>	<code>zipapp</code> — a 66 KB executable <code>.pyz</code>

There is also deliberately **no** `[build-system]` **table** in `pyproject.toml`. A build backend such as `setuptools` is a dependency too, and omitting it makes the empty manifest unambiguous at a glance.

8. How we prove zero dependencies

A manifest describes *intent*. `make prove` checks *reality*:

```
$ make prove
python          : 3.14.4 (297 stdlib modules known)
files scanned   : 9
ast nodes walked : 15,000+
distinct imports : 24
stdlib imports  : 23 / 24
first-party imports : 1 (barecode)
third-party imports : 0
manifest        : empty
RESULT: ZERO THIRD-PARTY DEPENDENCIES — manifest empty, every import is stdlib
```

It walks every source file with `ast`, collects every top-level import, and asserts each is a member of `sys.stdlib_module_names` — the frozen set CPython builds *at compile time* listing exactly what ships in

the box. It also asserts `pyproject.toml` declares no dependencies and has no build-system table. Non-zero exit if either fails. There is deliberately **no allowlist**: an escape hatch would defeat the point. The proof script is itself subject to the proof.

This is stronger than `npm ls` printing "(empty)", because `npm ls` only reads the manifest while this reads the code. The same two checks also run in GitHub Actions on a clean runner on every push, on Python 3.13 and 3.14, with **no `pip install` step anywhere in the workflow**.

9. Security posture

BareCode's job is inspecting untrusted directories, so this matters.

- **Nothing is executed.** We never import from, or launch anything in, the audited environment. `.pth` files that execute code are detected and reported, never run.
- **Path containment.** `RECORD` is attacker-controllable in a poisoned package. A recorded path resolving outside the environment root is a critical `escapes-environment` finding, not something we follow.
- **No shelling out.** We never invoke `pip`, `git`, `uv` or any other process — both a hackathon rule and correct design.
- **No network.** Nothing to exfiltrate to, nothing to intercept.
- **Symlinks are not followed** when walking a project tree, avoiding loops and escapes.
- **Degrades rather than fails.** Missing `RECORD`, unreadable directories and malformed `METADATA` produce reported findings, not tracebacks.
- **No invented cryptography.** We verify SHA-256 digests the installer already computed.

10. Testing

79 tests, stdlib unittest, no plugins, 1.1 seconds. Fixtures are synthesised on disk — real `.dist-info` directories with real RECORD hashes in a `TemporaryDirectory` — so the suite is hermetic, offline, and never calls `pip`.

The tests that matter most are the ones covering real bugs we hit:

Test	Why it exists
<code>test_same_length_edit_detected_by_hash</code>	The attack that survives size and mtime checks. This is the product.
<code>test_console_script_outside_site_packages_is_not_a_finding</code>	A false positive we actually shipped and fixed: RECORD legitimately records <code>../../bin/foo</code> .
<code>test_quoted_csv_path_with_comma</code>	Fails if anyone replaces <code>csv.reader</code> with <code>split(",")</code> .
<code>test_transitive_dependency_is_not_phantom</code>	A false positive that would have made <code>deps</code> untrustworthy.
<code>test_rfc2047_encoded_header_does_not_crash</code>	Real METADATA files return a <code>Header</code> object, not a <code>str</code> .
<code>test_ast_ignores_module_names_in_strings_and_comments</code>	Proves we use the AST rather than a regex.
<code>test_cycle_does_not_hang</code>	Dependency cycles are legal in Python packaging and do occur.

11. Bonus challenges claimed

Bonus	Pts	Evidence
STDLIB Log	+3	<code>STDLIB.md</code> — 17 substitutions, each with a rationale and the limitation it inherits. Requirement is ≥ 10 .
Package Killer	+3	<code>PACKAGE-KILLER.md</code> — feature matrices against <code>pipdeptree</code> , <code>pip-licenses</code> and <code>deptr / pipreqs</code> , <i>including what they still do better</i> .
Reproducible Build	+5	<code>make repro</code> builds twice from a clean tree, asserts byte-identical output, prints both SHA-256 hashes. Scope per the FAQ: same machine, same toolchain.
Single File (declined)	+5	A single 3,000-line file would cost more under Code Quality & Idiom (25% of the score) than the +5 gains. Declared as declined in <code>.zero-dep.toml</code> rather than half-attempted.

12. Limitations we state openly

The rules say a naive-but-honest implementation outscores a fast one that hides its corners, so these are in the README, not buried:

- `.dist-info` **only** — legacy `.egg-info` installs are invisible to us.
- `RECORD` **coverage is partial by nature** — `RECORD` itself and `.pyc` files carry no hash, so roughly a third of entries cannot be verified. We print that count rather than implying a clean bill of health.
- **A rewritten** `RECORD` **defeats** `verify` — an attacker with write access can update the hashes to match their payload. We catch everything that alters files *without* also rewriting the manifest. Detecting that needs an out-of-band record.
- **No PEP 508 marker evaluation, no PEP 440 version comparison** — so no conflict detection. We record markers verbatim and label edges conditional rather than guessing, because a wrong dependency edge is worse than an absent one.
- **Lockfiles are not read as declarations** — `poetry.lock` / `uv.lock` list the full transitive closure, which would make every transitive package look intentional.
- `deps` **cannot see dynamic imports** — `importlib.import_module(computed_name)` is invisible to static analysis.
- **No CJK width handling** in column alignment.

13. Key numbers to remember

Lines of Python (src + tools + tests)	~2,600
Tests, all passing	79
Third-party dependencies	0
Distinct stdlib modules used	23
Package → stdlib substitutions documented	17
Artifact size (<code>dist/barecode.pyz</code>)	66 KB
<code>verify</code> on a real 190-package env	4,927 files hashed in 3.4 s
Build command	<code>make build</code> (one step, stdlib <code>zipapp</code>)

14. Likely judge questions, and the answers

"Isn't reading pip's files a hidden dependency on pip?"

No, and the FAQ rules on this explicitly. Invoking a separately installed tool at runtime is forbidden; *parsing files those tools already produced* is permitted, on two conditions — disclose it in `STDLIB.md` (we do, §14) and degrade gracefully when the files are absent (we report packages without `RECORD` as unverifiable rather than failing). We never execute `pip`.

"Why not just use `importlib.metadata` ? It's stdlib."

It resolves against the *running* interpreter. BareCode audits an arbitrary target environment, and doing that through `importlib.metadata` would mean putting a foreign `site-packages` on `sys.path` — importing code from the thing we are auditing. We read the on-disk layout instead.

"A rewritten RECORD defeats this, so what's the point?"

It raises the bar from "edit a file" to "edit a file and correctly rewrite the manifest", and it catches every attack class that does not bother — which historically is most of them. We state the limitation in the README rather than waiting to be asked.

"Is Bun/Node/Go not better for Track A?"

The cheat sheet says Node or Go are *easiest*, not required. For a multi-command CLI, `argparse` is strictly more capable than Node's `util.parseArgs`, which supports only string and boolean flags with no subcommands.